

סיכום סיבוכיות וחישוביות - תשפ"ו 2026

22 במרץ 2026

גיא יער-און

הסיכום נכתב במהלך סמסטר ב' תשפ"ו (2026), ומבוסס על הרצאותיה של פרופסור טלי קאופמן ומבוסס על הקלטות הרצאות 2025 של פרופסור דודי מס, לכן יתכן שנפלו טעויות בסיכום, כך שהשימוש בו על אחריותכם בלבד.

תוכן עניינים

1	הרצאה 1	1
2	1.1 בעיות חיפוש	1.1
2	1.1.1 דוגמאות חשובות לבעיות חיפוש	1.1.1
2	1.2 בעיות הכרעה	1.2
2	1.3 פתרון בעיות חיפוש והכרעה	1.3
3	1.4 פתרון יעיל וסיבוכיות זמן	1.4
3	1.5 מחלקות של בעיות חיפוש	1.5
4	1.6 מחלקות של בעיות הכרעה	1.6
5	1.7 תרגול 1	1.7
6	2 הרצאה 2	2
6	2.1 הקשר בין בעיות חיפוש לבעיות הכרעה	2.1
7	2.2 רדוקציות	2.2
7	2.3 רדוקציית קוק	2.3
8	2.4 רדוקציות קארפ	2.4
10	2.5 תרגול 2	2.5
10	3 הרצאה 3	3
10	4 הרצאה 4	4
10	5 הרצאה 5	5
10	6 הרצאה 6	6
10	7 הרצאה 7	7
10	8 הרצאה 8	8
10	9 הרצאה 9	9
10	10 הרצאה 10	10
10	11 הרצאה 11	11

1 הרצאה 1

בסיום הקורס "מודלים חישוביים" דנו בשאלה: "מה ניתן לחשב באמצעות מחשב?", וראינו שישנן בעיות כמו בעיית אימות תוכנה (ATM) שלא ניתנת לפתרון באמצעות מחשב, וראינו בעיות רבות נוספות שלא ניתנות לפתרון באמצעות מחשב. בקורס זה, נדון בשאלה: "מה אפשר לחשב באמצעות מחשב באופן יעיל?". כלומר, יתכן שהזמן שניתן לפתור בעיה מסוימת הוא אקספוננציאלי וזה לא באמת ישים לחישוב באמצעות מחשב (עבור $n \rightarrow \infty$). על יעילות אפשר להסתכל ממרחב היבטים - סיבוכיות זמן, מקום, כמה ביטים אקראיים ו/או אינטרקציה (כל אלו נקראים "משאבי חישוב").

מטרת הקורס: אילו בעיות ניתן לחשב באופן יעיל?
כעת, נרחיב על מושגים אלו: בעיה, חישוב, יעילות.

1.1 בעיות חיפוש

הגדרה 1. נגדיר בעיית חיפוש מהצורה הבאה: בהינתן בעיה, נדרש למצוא לה תשובה/פתרון.

לדוגמה: בהינתן גרף $G = (V, E)$ וזוג קודקודים $s, t \in V$, בעיית החיפוש המתאימה יכולה להיות: מצא את המסלול הקצר ביותר בין s ל- t .

הגדרה 2. נייצג בעיית חיפוש באמצעות אוסף של זוגות, כלומר: יחס $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$ כאשר זוג $(x, y) \in R$ אם y הינו פתרון חוקי עבור בעיה x .

לדוגמה:

$$R_{ST-conn} = \{((G, s, t), P) \mid G \text{ Graph, } P \text{ is a path from } s \text{ to } t \text{ in } G\}$$

הערה 3. אנחנו מייצגים את בעיית החיפוש באמצעות $\{0, 1\}^*$ כיוון שניתן לקודד בינארית כל קלט ופלט (בעיה ופתרון) לאפסים ואחדות.

1.1.1 דוגמאות חשובות לבעיות חיפוש

דוגמה 4. בעיית $3-COL$

קלט: נתון גרף $G = (V, E)$

פלט: פונקציית צביעה $\phi : V \rightarrow \{1, 2, 3\}$ ללא קשת מונוכרומטית (ללא קשת ששני קודקודיה צבועים באותה צבע), אם קיימת כזו. פורמלית:

$$R_{3-COL} = \{(G = (V, E), \phi) \mid G \text{ is Graph, } \phi : V \rightarrow \{1, 2, 3\}\}$$

דוגמה 5. בעיית SAT

קלט: נתונה נוסחה בוליאנית בצורת $3CNF$, נזכר כי צורת CNF היא כזו שבטוויי "וגם" מפרידים בין פסוקיות שונות, כל חלק של סוגריים יקרא פסוקית ובתוך כל פסוקית ישנם משתנים בוליאניים (ליטרלים) מופרדים עם סימן $\vee$ בלבד ויכולים להופיע מפורשות או בצורת השלילה שלהם. $3CNF$ היא נוסחת CNF כנ"ל בה בכל פסוקית יש בדיוק 3 משתנים. למשל: $(X_1 \vee X_2 \vee \overline{X_3}) \wedge (X_1 \vee X_2 \vee X_4) \wedge (X_7 \vee \overline{X_2} \vee \overline{X_{90}})$.
פלט: פונקציית השמה למשתנים שנותנת השמה מספקת (כלומר, ערך הביטוי הבוליאני בצורת $3CNF$ יהיה $True$), אם קיימת כזו.

1.2 בעיות הכרעה

הגדרה 6. בעיית הכרעה: בהינתן קלט כלשהו, נדרש להכריע האם הוא מקיים תכונה כלשהי. התשובה היא בינארית: כלומר, כן או לא.

לדוגמה:

1. האם בהינתן $G = (V, E)$ גרף, G קשיר?

2. האם בהינתן $G = (V, E)$ וזוג קודקודים $s, t \in V$ קיים מסלול בין s ל- t ?

3. יהי $x \in \mathbb{N}$. האם x הוא מס' ראשוני?

הגדרה 7. מידול בעיית הכרעה: נתאר בעיית הכרעה בעזרת קבוצה S באשר $x \in S$ אמ"מ x מקיים את התכונה שאנחנו מעוניינים בה π . ולכן, בהינתן קלט ההכרעה היא האם $x \in S$ (מקיים, כלומר התשובה היא כן) או שלא, ואז $x \notin S$.

לדוגמה:

$$S_{ST-conn} = \{(G, s, t) \mid G \text{ Graph, } \exists \text{ path from } s \text{ to } t \text{ in } G\}$$

1.3 פתרון בעיות חיפוש והכרעה

על מנת לדון פורמלית בפתרון בעיות חיפוש והכרעה, נפתח בהגדרה רשמית של מושג האלגוריתם.

הגדרה 8. אלגוריתם הוא אוסף סופי של כללים חישוביים שמגדירים תהליך חישוב במודל "סביר". המודל ה"סביר" בו נעסוק בקורס הוא מכונת טיורינג עם סרט אחד, מכונה שעוצרת על כל קלט.

הגדרה 9. עבור בעיית חיפוש R וקלט x נסמן ב- $R(x)$ את קבוצת הפתרונות החוקיים עבור x . כלומר,

$$R(x) = \{y | (x, y) \in R\}$$

כעת, נאמר כי אלגוריתם A פותר את בעיית החיפוש R אם מתקיימים התנאים הבאים לכל $x \in \{0, 1\}^*$:

1. אם $R(x) \neq \emptyset$ (ל- x יש איזשהו פתרון ביחס), אזי $A(x) \in R(x)$ (א-יחזיר פתרון כלשהו, אם יש כמה יחזיר שרירותי).
2. אם $R(x) = \emptyset$, אזי $A(x) = \perp$.

הערה 10. הסימון $\perp$ הוא סימון מוסכם לכך שאלגוריתם מחזיר "אין פתרון".

הגדרה 11. נאמר כי אלגוריתם A פותר בעיית הכרעה S אם לכל x מתקיים:

1. אם $x \in S$ אזי $A(x) = 1$
2. אם $x \notin S$ אזי $A(x) = 0$

1.4 פתרון יעיל וסיבוכיות זמן

הגדרה 12. תהי M מכונת טיורינג בעלת סרט בודד. עבור קלט כלשהו x נסמן ב- $t_M(x)$ את מספר הצעדים ש- M מבצעת בריצה על x , בהנחה ש- M אכן עוצרת על x (אם לא עוצרת נאמר כי $t_M(x) = \infty$). **סיבוכיות הזמן** של M הינה פונקציה $T_M : \mathbb{N} \rightarrow \mathbb{N}$ שמוגדרת ע"י $T_M(n) = \max_{x \in \{0,1\}^n} \{t_M(x)\}$. כלומר, מס' הצעדים המקסימלי ש- M מבצעת על קלט באורך n .

הערה 13. $\max$ מגיע על מנת לתאר את worst case שהקלט שלנו יכול להגיע אליו, שהרי אנחנו תמיד מעוניינים להסתכל על המקרה הגרוע ביותר. לכן, עבור קלט ספציפי נסתכל על מס' הצעדים המקסימלי שהמכונה תבצע עליו, וזו סיבוכיות זמן הריצה.

הגדרה 14. נאמר כי מכונת טיורינג M הינה פולינומית אם קיים פולינום p כך שלכל n מתקיים: $T_M(n) \leq p(n)$.

הגדרה 15. נאמר כי בעיית חיפוש/הכרעה ניתנת לפתרון יעיל אם קיימת מכונת טיורינג פולינומית עם סרט בודד שפותרת אותה.

הערה 16. **מוטיבציה:** למה פתרון פולינומי הוא פתרון יעיל? פולינום חסום יחסית היטב ולא גדל מהר כמו אקספוננציאל, וכן ניתן להרכיב בעיות פולינומיות זו בזו, כלומר ניתן להשתמש באלגוריתם אחד בכמה וכמה אלגוריתמים פולינומיים ולקבל סה"כ אלגוריתם פולינומי. זה לא נכון עבור אלגוריתם לינארי, שהרי אם נרץ אותו בלולאה נקבל זמן ריבועי.

הערה 17. האם אנחנו מפסידים מכך שהדיון שלנו מתעסק במ"ט בעלת סרט בודד? כן. השפה הפשוטה הבאה מראה זאת:

$$S = \{xx | s \in \{0, 1\}^*\}$$

במכונת טיורינג עם שני סרטים, היינו מעתיקים את הקלט לסרט השני, ומוודאים באמצעות שני הסרטים באמצעות סריקה עד אמצע הקלט שאכן $xx \in S$, זה יעלה כמובן $O(n)$.
במכונת טיורינג בעלת סרט בודד לעומת זאת, נצטרך לבצע "הלוך חזור" על גבי הסרט, על מנת להשוות בכל שלב איבר בודד ולסמן ולחזור אחורה. זמן הריצה במקרה זה הוא $\Omega(n^2)$.
 כלומר: אנחנו מפסידים בזמן הריצה. אך, נזכר בטענה הבאה ממודלים חישוביים:

משפט 18. (התזה של צ'רץ טיורינג) כל בעיה שניתנת לפתרון במודל חישובי כלשהו סביר, ניתנת לפתרון באמצעות מכונת טיורינג בעלת סרט בודד.

ונראה תזה חזקה אף יותר:

משפט 19. (התזה של cobham-edmonds) אם ניתן לפתור בעיה כלשהי במודל חישובי "סביר" בזמן $T(n)$, אזי ניתן לפתור אותה באמצעות מכונת טיורינג בעלת סרט בודד בזמן $Poly(T(n))$.

מסקנה. למרות שמפסידים בזמן הריצה לפעמים במ"ט בעלת סרט בודד, הפער בין פתרון הבעיה במודל זה יהיה פולינומי בלבד (לפי משפט 17), וזה נחשב יעיל לפי מה שהגדרנו קודם.

1.5 מחלקות של בעיות חיפוש

ישנן בעיות כמו "בהינתן קבוצה S , מהם כל תתי הקבוצות של S ?" זו לא בעיה קשה, היא אפילו ממש קלה, אך רק הזמן שידרוש לי לכתוב את כל הקלט שלי ($2^{|S|}$) הוא אקספוננציאלי, ולכן זו אינה בעיה יעילה. לכן מראש, נתמקד בבעיות שאורך הפלט שלהן הוא פולינומי.

הגדרה 20. נאמר כי יחס R הינו חסום פולינומית אם קיים פולינום p כך שלכל זוג $(x, y) \in R$ מתקיים $|y| \leq P(x)$.

הגדרה 21. נגדיר את המחלקה $PF=Polynomial Find$:

$$PF = \{R \mid R \text{ חסום פולינומית, וקיים אלגוריתם פולינומי שפותר את } R\}$$

למשל, $R_{ST-Conn} \in PF$

הגדרה 22. נגדיר את המחלקה $PC=Polynomial Check$:

$PC = \{R \mid A(x, y) = 1 \iff (x, y) \in R\}$ שמקיים A פולינומי וקיים אלגוריתם פולינומי שפותר את x . למשל, $R_{3-Color} \in PC$ כי בהינתן צביעה כלומר, מחלקת הבעיות שקיים להן אלגוריתם פולינומי שיועד להכריע לכל x, y האם y פותר את x . למשל, $R_{3-Color} \in PC$ כי בהינתן צביעה ניתן לעבור על קשתות הגרף ולוודא כי הצביעה חוקית בזמן לינארי אפילו, ובפרט פולינומי, אך זו שאלה פתוחה האם $R_{3-color} \in PF$.

נרצה לשאול את השאלה, מהו היחס בין המחלקות הללו? האם $PC \subseteq PF$? למעשה, איננו יכולים להכריע. לצורך העניין, $R_{3-color} \in PC$ אך איננו יודעים האם $R_{3-color} \in PF$. אך, האם $PF \subseteq FC$? התשובה היא לא. יש בעיות שאנחנו יודעים לפתור בזמן יעיל - אך לא יכולים לוודא פתרון בזמן יעיל. האינטואיציה לכך, היא שעל מנת לפתור עלינו להחזיר פתרון בודד. על מנת להיות במחלקה PC עלינו לדעת להתמודד עם כל זוג שהוא, זו בעיה קצת יותר קשה פתאום.

משפט 23. $PF \not\subseteq FC$

הוכחה: על מנת להוכיח, נגדיר את היחס הבא (בעיית חיפוש) $R = R_1 \cup R_2$, באשר:
 $R_1 = \{(M, x) \mid x \in \{0, 1\}^*, \text{ והמכונה עוצרת בריצה על } x \text{ "עוצרת"}\}$
 $R_2 = \{(M, x) \mid x \in \{0, 1\}^*, \text{ כאשר תמיד ניתן להגיד על המכונה "לא יודע" | "לא יודע"}\}$
נטען כי $R \in PF$ לכן, נציג את האלגוריתם הבא: לכל קלט נחזיר "לא יודע", מדובר באלגוריתם פולינומי שרץ בזמן $O(1)$ שפותר את R . הוא פותר את הבעיה כיוון שבפרט הזוג $((M, x), \text{"לא יודע"})$ תמיד נותן פתרון חוקי ביחס ליחס R (שפכיל הרי את R_2). שהרי, אלגוריתם שפותר בעיית חיפוש צריך להחזיר פתרון חוקי ביחס ל R , הפתרון "לא יודע" אכן נחשב חוקי.
נטען כי $R \notin PC$: נ"ש כי קיים אלגוריתם A כך שבדק נכונות של פתרונות. נבנה אלגוריתם B שפותר את $HALT$ (בעיית העצירה) באופן הבא:

$$A((M, x), \text{"עוצרת"}) = B(M, x)$$

כיוון ש A בודק נכונות של פתרונות על R , הוא ידע לבדוק ולהכריע האם M עוצרת על x (שהרי זה חלק מ R). במילים אחרות, הכרחנו את אלגוריתם B באפעצות אלגוריתם A לוודא האם אכן M עוצרת על x . כלומר, B הוא אלגוריתם שפותר את $HALT$. בסתירה לכך ש $HALT$ לא כריעה. ■

הערה 24. האם $PC \subseteq PF$? זוהי שאלה פתוחה, בפרט זו ה-שאלה (שמיד נדון בה). כיום לא ידוע אם כל אלגוריתם שניתן לוידוא נכונות בזמן פולינומי גם ניתן לפתרון בזמן פולינומי.

1.6 מחלקות של בעיות הכרעה

הגדרה 25. נגדיר את המחלקה P :

$$P = \{S \mid S \text{ קיים אלגוריתם פולינומי שמכריע את } S\}$$

למשל, $S_{st-conn} \in P$ ע"י הרצת דייקסטרה שאם הוא החזיר מסלול הוא בפרט יודע להחזיר "כן" או "לא". כמו כן, לא ידוע האם $S_{3-color} \in P$.

נבחין כי הגדרת המקבילה ל PC היא מעט קשה בבעיות הכרעה, שהרי בבעיית הכרעה קיבלנו פתרון אפשרי ובדקנו אם הוא חוקי. כאן, בבעיות הכרעה אנחנו רק בודקים האם "כן" או לא, ולכן נעבוד קצת יותר קשה:

הגדרה 26. נאמר כי לבעיית הכרעה S יש **מערכת הוכחה יעילה** אם קיים אלגוריתם פולינומי V וקיים פולינום p כך שלכל x :

1. **שלמות:** אם $x \in S$ אזי "ישנה הוכחה לכך שהתשובה היא כן", כלומר: קיים y באורך $|y| \leq p(x)$ כך ש $V(x, y) = 1$.

2. **נאותות:** אם $x \notin S$ אזי לכל y מתקיים כי $V(x, y) = 0$.

הערה. לא מובטח כי נמצא את ההוכחה בצורה יעילה, רק שקיימת כזו שמוודה פתרון בזמן יעיל.

V יקרא **מוודא מסוג NP**. y נקרא עדות עבור x . y הוא עד לכך ש $x \in S$. אם נרחיב קצת יותר, אזי V מקבל קלט x והוכחה y באורך פולינומי לכך ש $x \in S$ (יתכן שההוכחה שגויה). אם $x \in S$ אזי תהיה איזושהי הוכחה y קצרה כך שנביא אותה ל V הוא יאשר שהיא נכונה, ואם $x \notin S$ אזי כל הוכחה שנביא ל V הוא ידע שהיא לא נכונה.

הגדרה 27. נגדיר את המחלקה NP : כל הבעיות שניתנות לוידוא בזמן פולינומי.

$$NP = \{S \mid S \text{ מוודא מסוג } NP\}$$

משפט 28. $P \subseteq NP$

הוכחה: תהי $S \in P$, ולכן קיים אלג' A פולינומי שמכריע את S . נגדיר מוודא עבור S כך: $\forall y : V(x, y) = A(x)$. אזי V אכן פולינומי, כיוון ש A פולינומי. וכן הוא מקיים את 2 התכונות של מוודא NP :

1. $x \in S$: לפי הגדרה, צריך שיהיה קיים עד פולינומי y כך שהמוודא V יחזיר 1. כיוון ש $x \in S$ והרי מדובר בבעיית הכרעה, $A(x) = 1$. מכאן,

לפי ההגדרה, $V(x, y) = A(x) = 1$, לכן לכל y שנבחר (לא משנה פי יהיה העד), V יחזיר 1.

2. $x \notin S$: לפי הגדרה, לכל עד y יהיה צריך להתקיים $V(x, y) = 0$. אם כן, כיוון ש $x \notin S$ הרי ש $A(x) = 0$, ולפי הגדרתנו לכל y יתקיים

$$V(x, y) = A(x) = 0 \text{ כנדרש.}$$

הערה 29. נשים לב כי לפי הגדרת בעיית הכרעה, אם קיים אלגוריתם פולינומי A שיועד להכריע את S , כלומר $S \in P$, הרי שאותו אלגוריתם יכול לשמש גם כמוודא את הפתרון בזמן פולינומי. כלומר, אם יש לי אלגוריתם שפותר את הבעיה (מכריע), הוא יודע גם לוודא ולכן אכן $P \subseteq NP$. זה לא נכון ב $PF \not\subseteq PC$ כיוון שכפי שאמרנו בעוד שצריך לוודא שקיים פתרון אחד על מנת להיות ב PF , על מנת להיות ב PC צריך לדעת להתמודד עם כל קלט אפשרי. כיוון שכאן ישנם 2 מקרים בדיק - הכרעה של כן, והכרעה של לא, אזי כן מתקיים $P \subseteq NP$. כמו כן, נשים לב כי אלגוריתם ב NP מקבל גם את ההוכחה לכך ש $x \in S$ ורק צריך להכריע אם היא נכונה בזמן פולינומי (שקול לכך שאלגוריתם ב PC קיבל פתרון ודרש להכריע נכונותו).

דוגמה: $S_3\text{-color} \in NP$. במקרה הזה הרי $x = G$, y הוא צביעה של הגרף ב 3 צבעים, כלומר y הוא פונקציית צביעה, והמוודא V הוא אלגוריתם שמקבל את x, y ומחזיר כן או לא. במקרה הזה: V יעבור על כל הקשתות $(a, b) \in E$ ויוודא כי $\phi(a) \neq \phi(b)$. אם כן, הוא יחזיר שכן. אם קיימת לפחות קשת אחת שזה לא נכון לגביה הוא יחזיר שלא. מדובר באלגוריתם פולינומי הרי שעלותו $O(|E|)$, לינארי, ולכן המוודא פולינומי, ולכן אכן $S_3\text{-color} \in NP$.

הערה 30. **האם** $NP \subseteq P$? שאלת מליון הדולר - שאלה פתוחה: כל התאוריה של מדעי המחשב מתפתחת סביב השאלה הזו. ההשערה הרווחת היא שלא.

1.7 תרגול 1

בקורס נגדיר זמן יעיל כזמן ריצה $O(n^i)$ כאשר i הוא קבוע. לצורך העניין - $O(n^{1000000000000})$ הוא זמן יעיל. כפי שנבין בהמשך הקורס, זו לא בעיה פשוטה לדעת האם אנחנו יכולים למצוא פתרון בזמן פולינומאלי.

בעיה 31. נתבונן ביחס הבא:

$$R_{4\text{-Clique}} = \{(G, S) | G \text{ Graph, } S \text{ clique and } |S|=4\}$$

כלומר, G גרף לא מכוון ו S קליקה בגודל 4 בגרף. הוכיחו כי $R_{4\text{-Clique}} \in PF$. **פתרון:** עלינו להראות למעשה אלגוריתם פולינומי שפותר את הבעיה. בתור התחלה, נבחין כי R אכן חסום פולינומית כי $|S| \leq |V| \leq |G|$. כעת, נתבונן באלג' הבא:

$$A(G)$$

1. לכל תת קבוצה S של 4 בקודקודים ב G בצע:

אם לכל $u, v \in S$ יש קשת בניהם, החזר את S .

2. החזר $\perp$.

האלגוריתם נאיבי למדי, אך יש להוכיח פורמלית את נכונותו. כלומר, נרצה להראות פורמלית כי A פותר את $R_{4\text{-Clique}}$. נבחין כי אם אין קליקה בגודל 4 בגרף, האלגוריתם אכן יצליח במשימתו כי על כל קבוצה באורך 4 שהוא יבדוק הוא לא ימצא קליקה, ואכן יחזיר $\perp$. לכן, נניח בשלייה כי קיימת קליקה בגרף נסמנה $S = \{x_0, x_1, x_2, x_3\}$ שהאלגוריתם לא מוצא. אזי, בעת שיעבור על הקבוצה S הנ"ל בשלב 1 הוא ימצא כי $(x_i, x_j) \in E$ $\forall i \neq j \in [3]$ ולכן יחזיר את S , בסתירה להנחה. כעת, זמן הריצה: נראה כי האלגוריתם מבצע $O(1)$ עבודה לכל קבוצה (בדיקת הקשתות שלה), ומס' תתי הקבוצות בגודל 4 הינו:

$$\binom{|V|}{4} = \frac{|V|!}{(|V|-4)! \cdot 4!} \leq |V|^4 = O(|G|^4)$$

מסקנה: $R_{4\text{-Clique}} \in PF$.

■

בעיה 32. נתבונן ביחס הבא:

$$R_{Clique} = \{(G, k, S) | G \text{ Graph, } S \text{ Clique and } |S|=k\}$$

הראו כי $R_{Clique} \in PC$

פתרון: באופן דומה, נראה כי מדובר ביחס חסום פולינומית. שהרי, $|S| = k \leq |V| \leq |G|$. כעת, נרצה להציג אלגוריתם שמקבל G, k, S ובדוק אם אכן S קליקה בגודל k :
1. ראשית, נבדוק כי $|S| = k$, אחרת נחזיר שלא.
2. לכל $u, v \in S$ נבדוק האם $(u, v) \in E$. אם נמצא זוג שלא, נחזיר לא. אחרת לבסוף: נחזיר כן.
האלגוריתם רץ בזמן ריצה $O(|S|^2) \leq O(|G|^2)$. מכאן: $R_{Clique} \in PC$.

■

הערה 33. בביור, האלגוריתם שהצגנו קודם לא יעבוד עבור R_{Clique} שכן זמן הריצה שלו יהיה $O(|V|^k) \leq O(|V|^{|V|})$. זהו לא אלגוריתם פולינומי - כי הוא תלוי בקלט. אך, זה שהאלג' הקודם לא עבד לא אומר שהבעיה הזו לא ניתנת לפתרון. בפרט, אנחנו לא יודעים היום אלגוריתם פולינומי עבור הבעיה. כלומר - האם $R_{Clique} \in PF$ זו שאלה פתוחה.

בעיה 34. נגדיר "קבוצה שולטת" כקבוצת קודקודים S כך שלכל קודקוד $u \in V$ מתקיים או $u \in S$ או של u יש שכן $v \in S$. ונגדיר את היחס:

$$\text{Dominating-Set} = \{(G, k) | G \text{ has a dominating set of size } k\}$$

הראו כי $\text{Dominating-Set} \in NP$.

פתרון:

אינטואיטיבית, הבעיה NP כיוון שבהינתן (G, k) נוכל לקבל כעדות את הקבוצה השולטת והמוודא V יבדוק שאכן היא קבוצה שולטת לפי ההגדרה.

נגדיר $V((G, k), y) = 1$ החזר y קבוצה שולטת בגודל k .

נבחין כי V פולינומית כיוון שהבדיקה תהיה פולינומית - למעשה לכל קודקוד בגרף נבדוק האם $v \in S$, אחרת נעבור על כל שכניו ונבדוק אם יש לו שכן ב- S . זמן הריצה הוא

$$O(\sum_{v \in V} \deg(v)) = O(|E|) \leq O(|V|^2) \leq O(|G|^2)$$

נבחין כי אם $(G, k) \in DS$ אזי קיים y כזה, המקיים $|y| \leq |G|$ עבורו $V((G, k), y) = 1$. אחרת, אם $(G, k) \notin DS$, לכל y יתקיים $V((G, k), y) = 0$ שהרי לא קיימת קבוצה שולטת.

■

2 הרצאה 2

2.1 הקשר בין בעיות חיפוש לבעיות הכרעה

משפט 35. $PC \subseteq PF \iff P = NP$

הוכחה:

$\implies$ נניח $PC \subseteq PF$. נוכיח $P = NP$. אנו יודעים כי $P \subseteq NP$. לכן נרצה להוכיח $NP \subseteq P$. תהי $S \in NP$ בעיה כלשהי, נרצה להראות $S \in P$. הרי לפי הגדרה, קיים פולינום P ואלגוריתם פולינומי V כך ש:

$$x \in S \iff \exists y : |y| \leq p(x) : V(x, y) = 1$$

מה הפער שלנו בין NP ל- P ? אנחנו יודעים בהינתן עדות, להכריע שייכות. אך איננו יודעים באופן כללי למצוא בצורה יעילה את העדות בהכרח. לכן, נעזר בהנחה שלנו $PC \subseteq PF$.

נגדיר את בעיית החיפוש ההבאה:

$$R_S = \{(x, y) | s.t. |y| \leq p(x) \wedge V(x, y) = 1\}$$

נטען כי $R_S \in PC$. ראשית, לפי ההגדרה R_S חסום פולינומית y חסום באורך של $p(x)$. שנית, האלגוריתם V הוא אלגוריתם שבדוק שייכות R_S . (הרי הוא יודע להכריע האם y הוא עדות של x , כלומר האם $(x, y) \in R_S$). לפי ההנחה שלנו, $R_S \in PC \subseteq PF$. כלומר, $R_S \in PF$. לכן, קיים אלגוריתם פולינומי A שפותר את R_S . כלומר, אם $R_S(x) \neq \emptyset$ אזי האלגו' מחזיר y כך ש- $(x, y) \in R_S$. אחרת, מחזיר $\perp$. כעת, נרצה להעזר באלגוריתם A על מנת להוכיח $S \in P$. נבנה אלגוריתם B כך שבהינתן קלט x פריץ את $A(x)$ ומחזיר 1 $\iff A$ החזיר תשובה שאינה $\perp$.

נבחין כי B פולינומי כי זמן הריצה שלו כזמן הריצה של האלגו' A שהוא פולינומי. נבחין כי B מכריע את S שכן:

$$x \in S \iff \text{לפי הגדרה, קיים } y \text{ המקיים } |y| \leq p(x) \text{ כך } V(x, y) = 1 \iff (x, y) \in R_S \iff R_S(x) \neq \emptyset \iff A(x) \neq \perp \iff B(x) = 1$$

כלומר, $B(x) = 1 \iff x \in S$

לכסוף, האלגוריתם B אכן מכריע את S לפי הגדרה, והוא פולינומי, כנדרש. מסקנה: $S \in P$.

$\Leftarrow$ נניח $P = NP$ ונוכיח $PC \subseteq PF$.

תהי $R \in PC$. נרצה להציג אלגוריתם פולינומי שפותר את R , כלומר $R \in PF$.

אם כן, $R \in PC$ ולכן קיים פולינום P כך שלכל $(x, y) \in R$ מתקיים: $|y| \leq P(x)$. בנוסף, קיים אלגוריתם פולינומי A שבדוק שייכות ליחס R .

בבירור, נצטרך להגדיר בעיית הכרעה ולהראות כי היא נמצאת ב- NP , ששווה ל- P . ולהתקדם משם. אינטואיטיבית, נרצה למצוא את ה- y כמעבר אחד אחר אחד אבל! מדובר בזמן ריצה אקספוננציאלי. לכן נרצה לגלות את ה- y באמצעות מעבר ביט ביט, כלומר נגלה לאט לאט את y כאשר כל פעם נסתכל על $prefix(y)$ שונה שלו. עוד קצת אינטואיציה: בדינו אלגוריתם פולינומי שבדק אם y פתרון תקין עבור x . כמו כן, נבחין כי על מנת להראות שייכות ל- NP אנחנו רק צריכים להראות שקיים מודא וקיימת עדות. לכן מגדירים יחס כפי שיתואר למטה, לפיו נכנסים רק $prefix(y)$. אנחנו יודעים מי יהיה העדות - y'' , ואנחנו יודעים גם מי המודא: A עצמו! שמסוגל פשוט לשרשר ולבדוק אם אכן השרשור נמצא ב- R .

נגדיר את בעיית ההכרעה הבאה:

$$S'_R = \{(x, y') | \exists y'' : (x, y' \circ y'') \in R\}$$

לדוגמה: $R = \{(10, 011)\}$ אזי $S'_R = \{(10, \varepsilon), (10, 0), (10, 01), (10, 011)\}$. כלומר - כל התחיליות של y .
נוכיח כי $S'_R \in NP$. מתקיים כי:

$$(x, y') \in S'_R \iff \exists y'' : |y''| \leq p(x + y') : A(x, y' y'') = 1$$

שהרי, לפי הגדרה העד יכול להיות y'' והמודא הוא האלגוריתם A שבדק שייכות ל- R , והרי יכול לשרשר $y' \circ y''$ ולבדוק אם ערך זה שייך ל- R .
כעת, לפי ההנחה, $P = NP$ ולכן גם $S'_R \in P$. כלומר, קיים אלגוריתם פולינומי D שמכריע את S'_R (מחזיר כן אם קיים y'' כנ"ל ששרשורו יהיה ב- R).

נגדיר אלגוריתם B שיפתור את R :

$$B(x)$$

א. ראשית - נבדוק אם בכלל קיים פתרון, לכן נריץ את $D(x, \varepsilon)$. אם הוא מחזיר כן: אזי בבירור קיים y'' כך $(x, y'') \in R$. אחרת - מחזיר לא: מחזיר $\perp$.

ב. אם הגענו לשלב הזה: בהכרח קיים פתרון, ונרצה לבנות אותו בצורה הדרגתית. נסמן $\varepsilon \rightarrow y$

ג. כל עוד $D(x, y) = 1$ (ה- y שאנו מסתכלים עליו עכשיו עודנו תחילית של פתרון):

1. אם $D(x, y1) = 1$ (שרשור של הביט 1 לסיום y עדיין נותן פתרון) אזי נגדיר $y \rightarrow y1$.

2. אחרת, אם $D(x, y0) = 1$ אזי $y \rightarrow y0$

נבחין כי האלגוריתם $B(x)$ הוא אלגוריתם פולינומי. בשלב 1 הוא מריץ אלגו' פולינומי D , בתוך הלולאה: הוא מריץ לכל היותר פעמיים אלגוריתם פולינומי. כמה פעמים מתבצעת הלולאה? לכל היותר אורך פולינומי של צעדים - האורך של y , ולכן זמן הריצה של הלולאה הוא מס' פולינומי של פעמים כפול ערך פולינומי: וסה"כ מדובר באלגוריתם פולינומי.

אם אין פתרון ל- x , בשלב 1 מחזיר $\perp$. אחרת, אם קיים פתרון, הוא נבנה רק בצורה לפיה הוא נשאר תחילית של פתרון: לכן בהכרח יחזיר y כך $(x, y) \in R$.

סה"כ B אלגוריתם פולינומי שמכריע את R , ולכן $R \in PF$.



מסקנה 36. ראינו מהמשפט הקודם שקילות בין בעיות פתוחות לגבי בעיות הכרעה ובעיות חיפוש, לכן נרשה לעצמנו בקורס להתעסק יותר בבעיות הכרעה.

הערה 37. בכיוון השני של הוכחת משפט 35, השתמשנו ברדוקציה. צמצמנו את הבעיה שלנו לבעיה אחרת: אם אנחנו יודעים לפתור (כאן, בצורה פולינומית ויעילה) בעיה מסויימת אזי (בגלל שפולינום כפול פולינום הוא פולינומי) ניתן לפתור את הבעיה השנייה בזמן יעיל. מה שמוביל אותנו לדבר על:

2.2 רדוקציות

הגדרה 38. רדוקציה הינה דרך לפתור בעיה אחת, בעזרת פתרון עבור בעיה אחרת. אם ישנה רדוקציה מבעיה A לבעיה B ואנחנו יודעים לפתור את B , אזי אנחנו יודעים לפתור את A .

הערה 39. קודם ביצענו רדוקציה מ- R ל- S'_R - בדינו באמצעות ההנחה היה פתרון ל- S'_R ובאמצעותו בנינו אלגוריתם B ששימש מעבר לרדוקציה אל R .

2.3 רדוקציית קוק

הגדרה 40. רדוקציית קוק מבעיה A לבעיה B הינה אלגוריתם פולינומי שפותר את A בעזרת גישת אוקרל ל- B . כלומר, האלגוריתם שפותר את A יכול לשאול את האוקרל שאלות עבור B , כאשר זמן הריצה עבור כל גישת אוקרל נחשבת כצעד אחד בודד. נסמן זאת ב- $A \leq_T^P B$ (מייצג פולינומית ו- T מייצג מכונת טיורינג. $\leq$ מתאר את הרדוקציה עצמה - A לא קשה יותר מ- B).

האורקל הוא מעין "קופסה שחורה" קסומה אשר ניתן לשאול אותה שאלות לגבי בעיה מסויימת, היא מחזירה תשובה נכונה באופן מיידי בצעד חישובי אחד מבלי שנדע כיצד הגיעה לפתרון.

הגדרה 41. רדוקציה עצמית: עבור בעיית חיפוש R נסמן ב- S_R את בעיית ההכרעה הבאה:

$$S_R = \{x | R(x) \neq \emptyset\}$$

אינטואיטיבית, לכל בעיית חיפוש ישנה בעיית הכרעה מתאימה לה - בעיית קדם: בהינתן x , האם קיים לה איזה שהוא y ? **רדוקציה עצמית** הינה רדוקציית קוק R ל- S_R . כלומר: $R \leq_P^P S_R$. (כלומר, מאפשר לפתור בעיית חיפוש באמצעות גישות לאורקל שפותר את בעיית ההכרעה).

מסקנה 42. אם יש לנו בעיית חיפוש קשה, אבל יש רדוקציה עצמית לבעיה זו. אזי נוכל לפתור את בעיית ההכרעה המתאימה, ובגלל שקיימת רדוקציה עצמית אזי פתרנו גם את בעיית ההכרעה.

דוגמה 43. נתבונן בדוגמה ראשונה לרדוקציה עצמית. נגדיר את בעיית החיפוש הבאה:

$$R_{SAT} = \{(\Phi, \tau) | \Phi \text{ is a Boolean formula in CNF, and } \tau \text{ is a satisfying assignment } \phi \text{ for the formula}\}$$

נראה רדוקציה עצמית עבור R_{SAT} .

אינטואיציה: נתחיל מלהציב למשל $x_1 = T$, ונקבל נוסחה בוליאנית מצורת CNF מצומצמת יותר. כעת נשאל את האורקל: האם הנוסחה שבידי היא ספיקה? אם כן: מצאנו $x_1 = T$ הוא "תחילית" של פתרון ונתקדם. אחרת: נציב $x_1 = F$ ונתקדם בכל מקרה לנוסחה.

נציג את האלגוריתם הבא:

$$A^{SAT}(\Phi)$$

1. שאל את האורקל אם $\Phi \in SAT$, אם לא נחזיר $\perp$.

2. נסמן n כמס' המשתנים בנוסחה Φ . כלומר, $\{x_i\}_{i=1}^n$ הם המשתנים ונגדיר $\varepsilon \rightarrow \tau$

3. לכל i מ 1 עד n :

א. הגדר נוסחה חדשה: $\Phi' = \tau T$ (למשתנה הבא, הצב ערך T) ושאל את האורקל אם Φ' ספיקה. אם $\Phi' \in SAT$ נגדיר $\tau = \tau T$

ואחרת נגדיר $\tau = \tau F$.

בבירור, האלגוריתם בונה את τ צעד אחר צעד ואם אכן קיימת השמה מספקת אזי הוא מחזיר אחת שכזו τ , ואם אין כזו מנכונות האורקל הוא מחזיר בשלב 1 $\perp$.

זמן הריצה: כמובן מריצים את האורקל n פעמים, ולכן הפער פולינומי.

■

הערה 44. הערה מגניבה. אנחנו יודעים כי $R_{SAT} \in NP$, קל לבדוק בהינתן נוסחה האם היא אכן ספיקה. אך לא ידוע אם היא ב- P , כלומר האם קיים אלג' פולינומי שיועד לומר: יש/אין פתרון. אם $P = NP$, אזי קיים אלג' פולינומי שיועד לומר - יש/אין פתרון: ומהשקילות שראינו כאן עם רדוקציית קוק עצמית, נריץ אותו n פעמים, סה"כ פולינומי כפול n נותר פולינומי ולכן $R_{SAT} \in P$.

הערה 45. כסימון, $A^{SAT}(\Phi)$ מסמן שהאלגוריתם A מקבל גישות אורקל לבעיית SAT . כקונבציה, כך נסמן. כלומר: האלגוריתם A מסוגל לגשת בגישת אורקל אל בעיית ההכרעה.

הערה 46. **רדוקציית קוק** רלוונטית הן לבעיות הכרעה והן לבעיות חיפוש. כאשר נעבור לדון בסוג של רדוקציה שרלוונטי רק לבעיות הכרעה:

2.4 רדוקציות קארפ

הגדרה 47. רדוקציית קארפ מבעיית הכרעה S_1 לבעיית הכרעה S_2 הינה פונקציה **שניתנת לחישוב בזמן פולינומי ומקיימת**: $x \in S_1 \iff f(x) \in S_2$. במקרה זה, נסמן $S_1 \leq_P^P S_2$ (P מייצג פולינומי ו- m מייצג many to one: היא לא חח"ע, יתכן שיהיו שני ערכים $x_1 \neq x_2$ כך ש- $f(x_1) = f(x_2)$ כמו כן היא לא צריכה להיות על).

הערה 48. הרדוקציות בהן התעסקנו בקורס "**מודלים חישוביים**" הן רדוקציות קארפ.

הערה 49. הפונקציה לא הפיכה בהכרח כפי שאמרנו - יתכן והפונקציה (רדוקציה) הינה $S_1 \leq_m^P S_2$, כך שבהינתן קלט ב- S_1 ניתן להמירו לקלט ב- S_2 אך הכיוון ההפוך אינו נכון!

דוגמה 50. (דוגמה חשובה). נתבונן בדוגמה ראשונה לרדוקציית קארפ. נזכר בבעיית SAT ונגדיר את הבעיה הבאה:

$$IS = \{(G, k) | G \text{ Graph, } G \text{ has independent set of size at least } k\}$$

נרצה להראות רדוקציית קארפ $SAT \leq_m^P IS$.

מה אנחנו רוצים להראות? בהינתן נוסחה בוליאנית ϕ בצורת CNF כקלט לבעיית SAT , נרצה לבנות גרף G ולהגדיר מס' k כך שהבנייה תהיה בזמן פולינומי, כך שיתקיים: $\phi \in SAT \iff (G, k) \in IS$. כלומר בשלב ראשון נראה את הבנייה, נוכיח שהיא פולינומית. בשלב שני נוכיח את השקילות הנ"ל.

נסמן: $\phi = \bigwedge_{i=1}^m (\bigvee_{j=1}^{n_i} \ell_{i,j})$ כנוסחה הבוליאנית בצורת CNF , כך שעבור הפסוקית i יש בו n_i ליטרלים מופרדים $\forall$ כך ש $n_{i,j} \in \{x_r, \overline{x_r}\}$ עבור $r \in [n]$ כך שאנו יודעים שהנוסחה מורכבת מהמשתנים $\{x_i\}_{i=1}^n$. נגדיר $G = (V, E)$ כך:

$$V = \{u_{i,j} | \forall 1 \leq i \leq m, 1 \leq j \leq n_i : \ell_{i,j} = u_{i,j}\}$$

כלומר, כל ליטרל מיוצג ע"י קודקוד. נגדיר את הקשתות E כך:

1. סוג ראשון של קשתות: לכל פסוקית $1 \leq i \leq m$ נרצה להוסיף קשתות בין כל הקודקודים של אותה הפסוקית, כך שהם יהוו קליקה. כלומר:

$$E_1 = \{(u_{i,j}, u_{i,k}) | \forall 1 \leq i \leq m, \forall j \neq k : j, k \in [n_i]\}$$

2. סוג שני של קשתות: עבור על משתנה $x_r \in \phi$ כך ש $r \in [n]$ נוסיף קשתות בין כל קודקוד שמתאים ל x_r לבין כל קודקוד שמתאים ל $\overline{x_r}$. כך:

$$E_2 = \{(u_{i,j}, u_{a,b}) | \ell_{i,j} = \overline{\ell_{a,b}}\}$$

וסה"כ $E = E_1 \cup E_2$.

כמו כן, נגדיר $k = m$. כלומר k הוא מס' הפסוקיות. עד לשלב זה הייתה הבנייה של הגרף G , ונבחין שהיא אכן פולינומית (אין צורך ממש לחשב בכמה זמן), אך בבירור זה פולינומי. כעת נרצה להראות כי מתקיים $\phi \in SAT \iff (G, k) \in IS$.

אינטואיטיבית, הרדוקציה הנ"ל בונה משחק שבו כדי לנצח חייבים לספק את הנוסחה. בנוסחת CNF על מנת שהיא תהיה אמת כל פסוקית חייבת לקבל ערך של אמת. כדי שפסוקית תהיה אמת, מספיק שליטרל אחד יהיה אמת בתוכה. בגרף: כל פסוקית הפכה לקליקה. נבחין כי ב IS , אפשר לבחור לכל היותר קודקוד אחד מתוך כל קליקה כזו. כיוון שהגדרנו $k = m$, אנחנו למעשה "מכריחים" את האלגוריתם לבחור בדיוק נציג אחד מכל פסוקית, הבחירה הזו שקולה להחלטה: "זה הליטרל שישפך פסוקית זו". בנוסף, חיברנו כל משתנה לשאר מופעיו בצורת השלילה שמופעים בפסוקיות אחרות. מדוע? על מנת למנוע מצב בו הגדרנו $x_1 = T, \overline{x_1} = T$ כעת, נפרמל אינטואיציה זו:

$\implies$ נניח כי $\phi \in SAT$. אזי, קיימת השמה τ שמספקת את ϕ . נרצה להראות כי $(G, k) \in IS$. כלומר, בגרף זה ישנה קבוצה ב"ת מגודל לפחות k . אנו יודעים כי τ מספקת את ϕ ולכן בכל פסוקית יש ליטרל אחד לפחות $\ell_{i,j}$ שמספק אותה. נטען כי הקבוצה הבאה: $S = \{u_{i,j}\}_{i=1}^m$ המקיימת $|S| = m = k$ היא קבוצה ב"ת ב G . נבחין כי בין כל זוג קודקודים שכאלו אין קשת כיוון שהם מפסוקיות אחרות, פרט למקרה היחיד שיש קשת בין פסוקיות שונות ובו קיים משתנה שגם הוא מספק את הפסוקית, וגם השלילה שלו מספק את הפסוקית. שהרי במקרה זה אם נב"ש שאכן קיים כזה אזי $\ell_{i,j} = T$ וגם $\ell_{a,b} = T$ אבל $\ell_{a,b} = \overline{x_r} \wedge \ell_{i,j} = x_r$ ונקבל סה"כ $\ell_{a,b} = F$ בסתירה. ולכן: אכן הקבוצה הנ"ל היא קבוצה ב"ת ב G בגודל k , כנדרש.

$\Leftarrow$ נניח כי $(G, k) \in IS$. אזי, קיימת G קבוצה ב"ת בגודל k . נרצה להגדיר השמה τ לנוסחה ϕ ולהוכיח כי היא אכן מספקת את ϕ . נגדיר השמה τ ϕ בהתאם לקבוצה S , כך: אם $u_{i,j} \in S$ וכן $u_{i,j} = x_r$ אזי נגדיר $\tau(x_r) = T$ אם $u_{i,j} = \overline{x_r}$ אזי נגדיר $\tau(x_r) = F$. נטען כי בהכרח ההשמה הנ"ל מוגדרת היטב, שכן לא יתכן כי x_r קיבל גם T וגם F . בשלילה שכן, אזי קיים x_r עבורו הייתה השמה T , ולכן $u_{i,j} \in S$ וגם קיימת השמה F כלומר $u_{a,b} \in S$. אך קודקודים אלו מייצגים אותם קודקודים - ולכן ישנה קשת $(u_{i,j}, u_{a,b}) \in E$ בסתירה לכך ש S היא קבוצה ב"ת. מסקנה: ההשמה הנ"ל מוגדרת היטב.

כעת, נרצה להוכיח שאכן לפי ההשמה τ ϕ כעת היא ספיקה. כלומר, $\tau(\phi) = T$. אנו יודעים כי $|S| = k$, וכן לא יכולים להיות זוג קודקודים מאותה פסוקית (שכן אחרת ישנה קשת בניהם, כי כל פסוקית היא קליקה ב"ת), לכן בהכרח S מכילה קודקוד אחד מכל פסוקית: אותו ליטרל שמתאים לפסוקית הנ"ל ונמצא ב S מספק את הפסוקית המתאימה לו, וסה"כ כל הפסוקיות מסופקות: לכן ϕ מסופקת ע"י τ , כנדרש.

2.5	תרגול 2
3	הרצאה 3
4	הרצאה 4
5	הרצאה 5
6	הרצאה 6
7	הרצאה 7
8	הרצאה 8
9	הרצאה 9
10	הרצאה 10
11	הרצאה 11